EVALUACION 03

Gestion de pedidos de restaurante

Jesus Jose Castillo Sumire

Curso: Desarrollo de Aplicaciones Web

Noviembre de 2026

Objetivo

Construir una API distribuida en tres servicios Spring Boot para administrar clientes, platos y pedidos. Cada servicio conserva sus datos en una base MySQL independiente; el servicio de pedidos consulta a Cliente y Plato por REST para validar referencias y calcular importes.

Como se trabajo

Organice el proyecto en tres modulos Maven. Defini entidades JPA, repositorios y capas de servicio/controlador. Configure perfiles YAML de desarrollo y produccion, conecte MySQL de XAMPP y ejecute la coleccion de pruebas desde Postman. La evidencia visual de este informe proviene de Postman, XAMPP, phpMyAdmin y Visual Studio Code.

Resultado comprobable

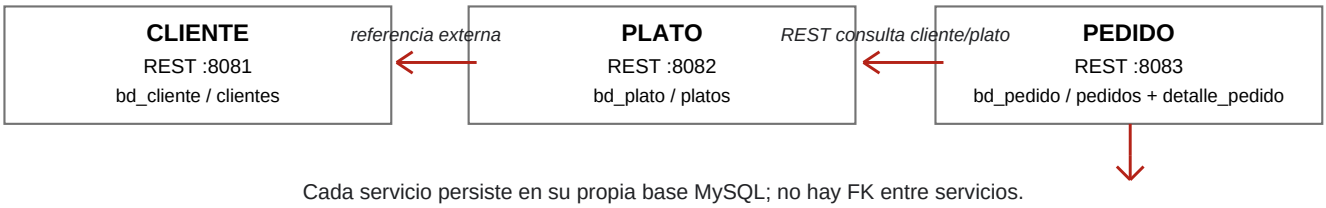
Postman: 28 pruebas, 28 aprobadas, 0 fallidas y 0 errores. **Persistencia:** tres bases separadas; tablas visibles en phpMyAdmin. **Proyecto:** tres servicios y sus carpetas se ven en VS Code.

Asignacion individual

El PDF Evaluacion 03 - D asigna a CASTILLO SUMIRE, JESUS JOSE el laboratorio individual N. 4: Pedidos de restaurante, con los microservicios Cliente, Plato y Pedido. Este proyecto y sus evidencias corresponden a esa asignacion.

Arquitectura y flujo

Los tres servicios se ejecutan de forma independiente. Cliente y Plato publican sus operaciones CRUD; Pedido almacena las solicitudes y sus detalles. Las referencias del cliente y del plato son identificadores remotos, no claves foraneas entre bases diferentes.



Cada servicio persiste en su propia base MySQL; no hay FK entre servicios.
Pedido valida cliente y platos, toma el precio vigente y calcula subtotales y total.

Flujo principal del pedido

1. Postman envia el pedido a Pedido (puerto 8083). 2. Pedido consulta Cliente (8081) y verifica que exista y este activo. 3. Pedido consulta cada Plato (8082), comprueba disponibilidad y lee el precio vigente. 4. Calcula subtotal = precio x cantidad y suma los subtotales. 5. Guarda pedido y detalles en bd_pedido y devuelve la respuesta REST.

Servicios y bases de datos

Servicio	Puerto	Base de datos / tablas
Cliente	8081	bd_cliente / clientes
Plato	8082	bd_plato / platos
Pedido	8083	bd_pedido / pedidos, detalle_pedido

La aplicacion no necesita Apache para servir sus APIs: Spring Boot atiende en 8081, 8082 y 8083. XAMPP se utiliza aqui para ejecutar MariaDB/MySQL en el puerto 3306.

Implementacion tecnica

Cada servicio mantiene la separacion Controller - Service - Repository. Los controladores reciben solicitudes HTTP, los servicios aplican reglas de negocio y los repositorios usan Spring Data JPA/Hibernate para guardar y consultar entidades. DTO separan los datos recibidos y devueltos del modelo persistente.

Pedido y sus detalles

La entidad Pedido se relaciona con sus detalles con @OneToMany; DetallePedido referencia al pedido con @ManyToOne. Los campos idCliente e idPlato conservan referencias a otros servicios como valores, sin relacion JPA entre bases.

Fragmento literal de PedidoService - consulta y calculo

```
BigDecimal subtotal = plato.precio()
    .multiply(BigDecimal.valueOf(solicitud.cantidad()));
DetallePedido detalle = new DetallePedido();
detalle.setIdPlato(plato.idPlato());
detalle.setNombrePlato(plato.nombre());
detalle.setCantidad(solicitud.cantidad());
detalle.setPrecioUnitario(plato.precio());
detalle.setSubtotal(subtotal);
pedido.addDetalle(detalle);
total = total.add(subtotal);
pedido.setTotal(total);
```

Este bloque usa el precio que responde el servicio Plato, persiste el detalle y acumula el total. La clase PedidoService tambien rechaza platos repetidos dentro de un mismo pedido, platos no disponibles y referencias inexistentes.

Configuracion de base y perfiles

En dev, cada URL JDBC incluye createDatabaseIfNotExist=true y Hibernate usa ddl-auto=update. Esto permite crear la base al arrancar cuando MySQL da permiso y crear/actualizar tablas. En prod se requieren variables de entorno y ddl-auto=validate, para comprobar el esquema sin alterarlo.

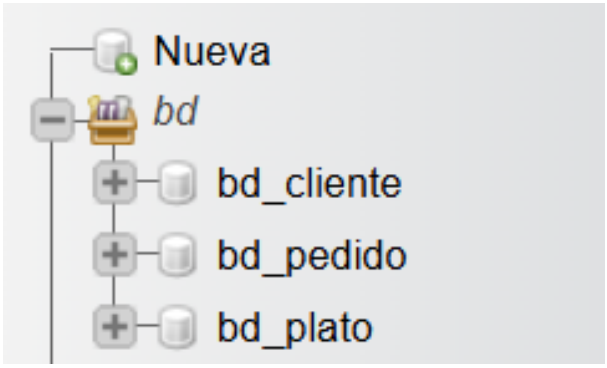
Arranque local

```
# En tres terminales desde la carpeta del proyecto
.\mvnw.cmd -pl cliente-service spring-boot:run
.\mvnw.cmd -pl plato-service spring-boot:run
.\mvnw.cmd -pl pedido-service spring-boot:run
```

Herramientas: Java 17, Spring Boot, Spring Web, Spring Data JPA, Hibernate, MySQL/MariaDB de XAMPP, Maven Wrapper, Visual Studio Code y Postman.

Evidencia de persistencia - phpMyAdmin

La captura del arbol de bases muestra bd_cliente, bd_pedido y bd_plato. Las vistas de tablas confirman clientes (1 fila), platos (2 filas), pedidos (1 fila) y detalle_pedido (2 filas) en el momento de las capturas.



Captura real de phpMyAdmin: bases creadas para los tres servicios.

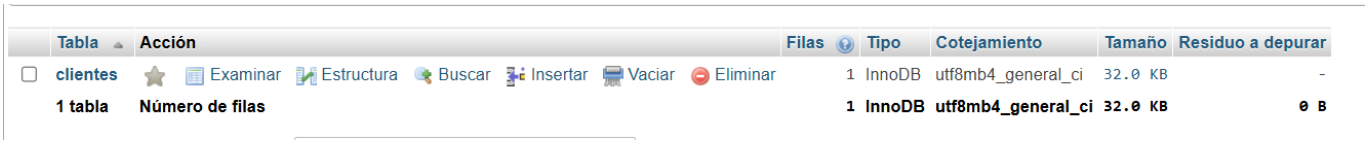


	Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐	clientes	Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_general_ci	32.0 KB	-
	1 tabla	Número de filas	1	InnoDB	utf8mb4_general_ci	32.0 KB	0 B

bd_cliente: tabla clientes, con 1 registro visible.

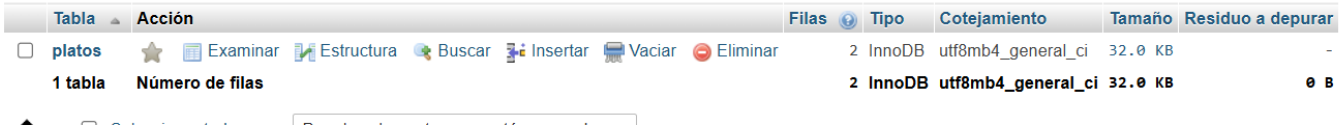


	Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐	platos	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_general_ci	32.0 KB	-
	1 tabla	Número de filas	2	InnoDB	utf8mb4_general_ci	32.0 KB	0 B

bd_plato: tabla platos, con 2 registros visibles.

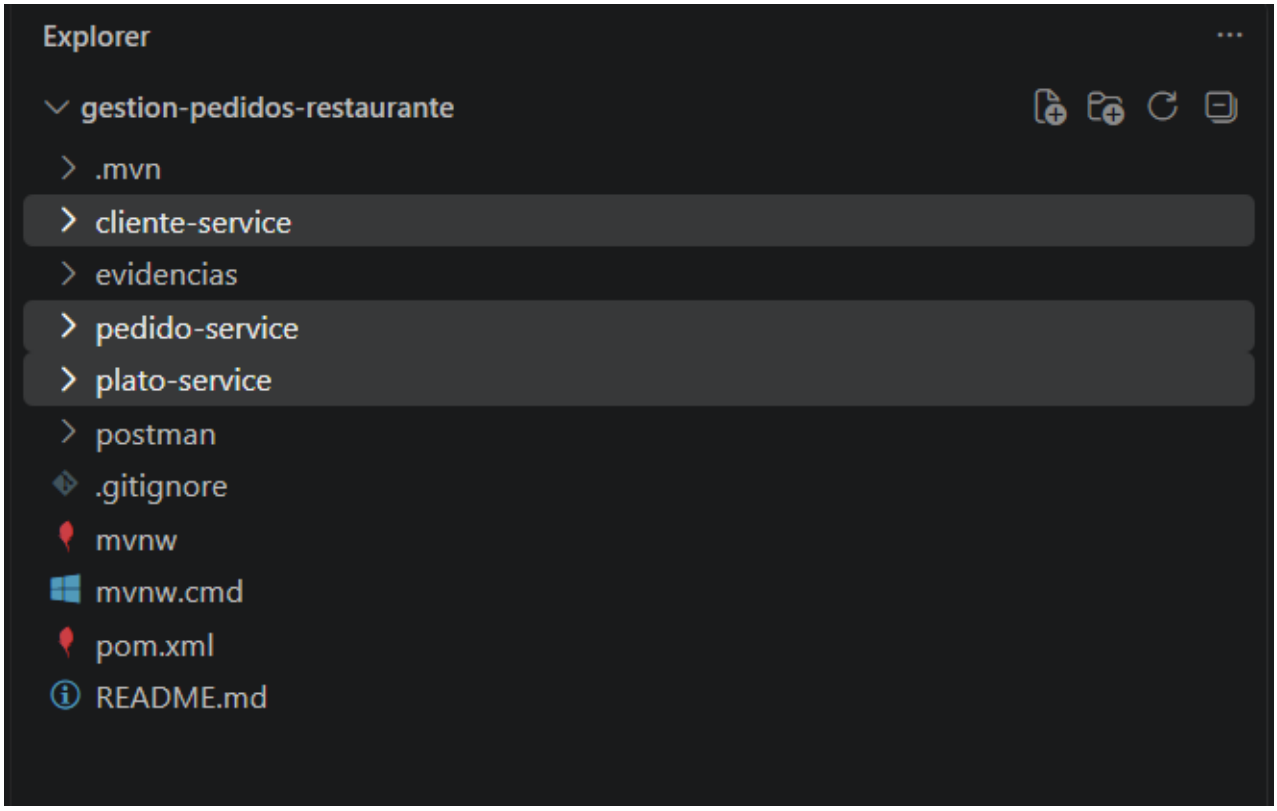


	Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐	detalle_pedido	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_general_ci	32.0 KB	-
☐	pedidos	Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_general_ci	16.0 KB	-
	2 tablas	Número de filas	3	InnoDB	utf8mb4_general_ci	48.0 KB	0 B

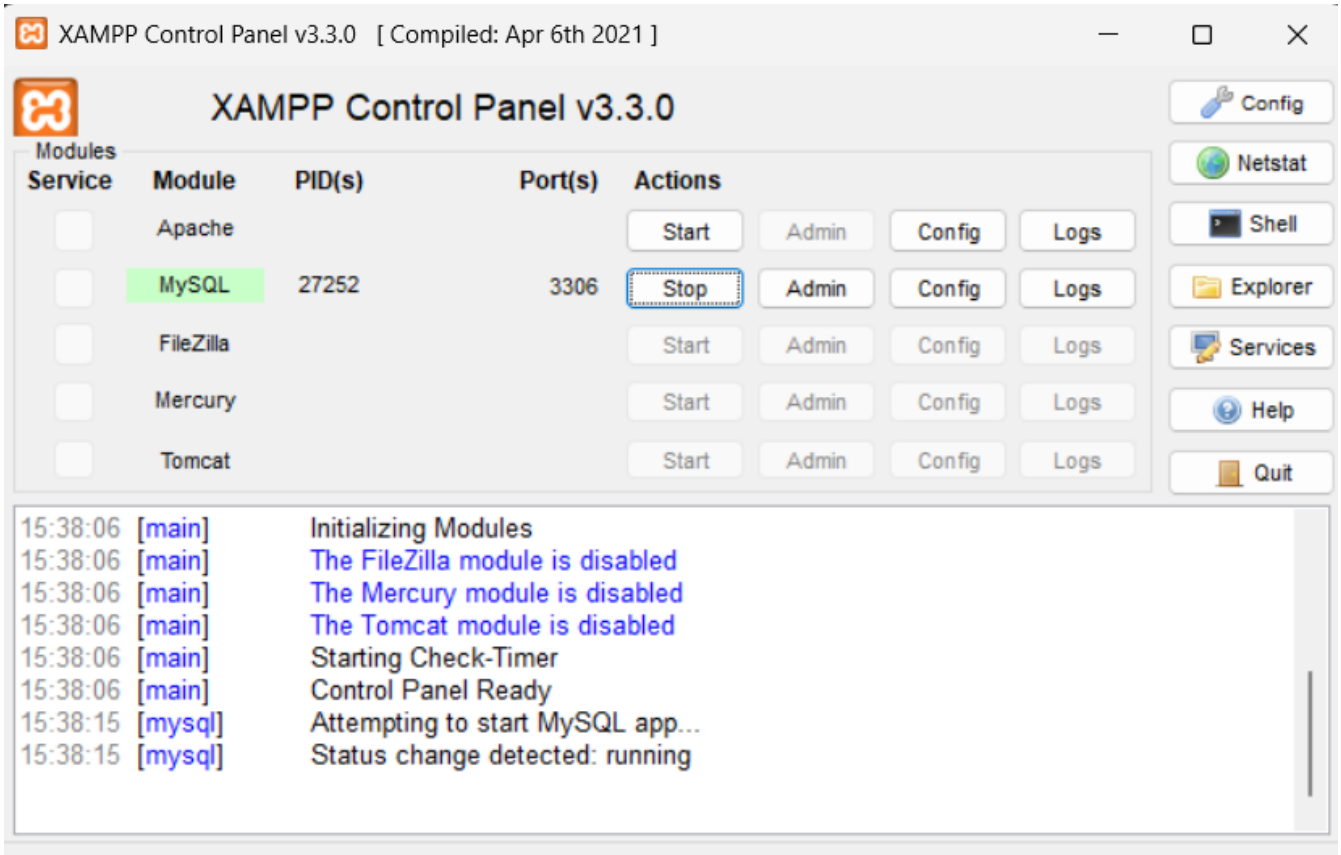
bd_pedido: pedidos y detalle_pedido, con 1 pedido y 2 detalles.

Evidencia del entorno de trabajo

La captura de VS Code muestra la carpeta del proyecto y los tres modulos. La captura de XAMPP muestra MariaDB/MySQL activo en el puerto 3306. La ejecucion de las APIs tambien queda demostrada por las respuestas exitosas a localhost:8081, 8082 y 8083 en Postman.



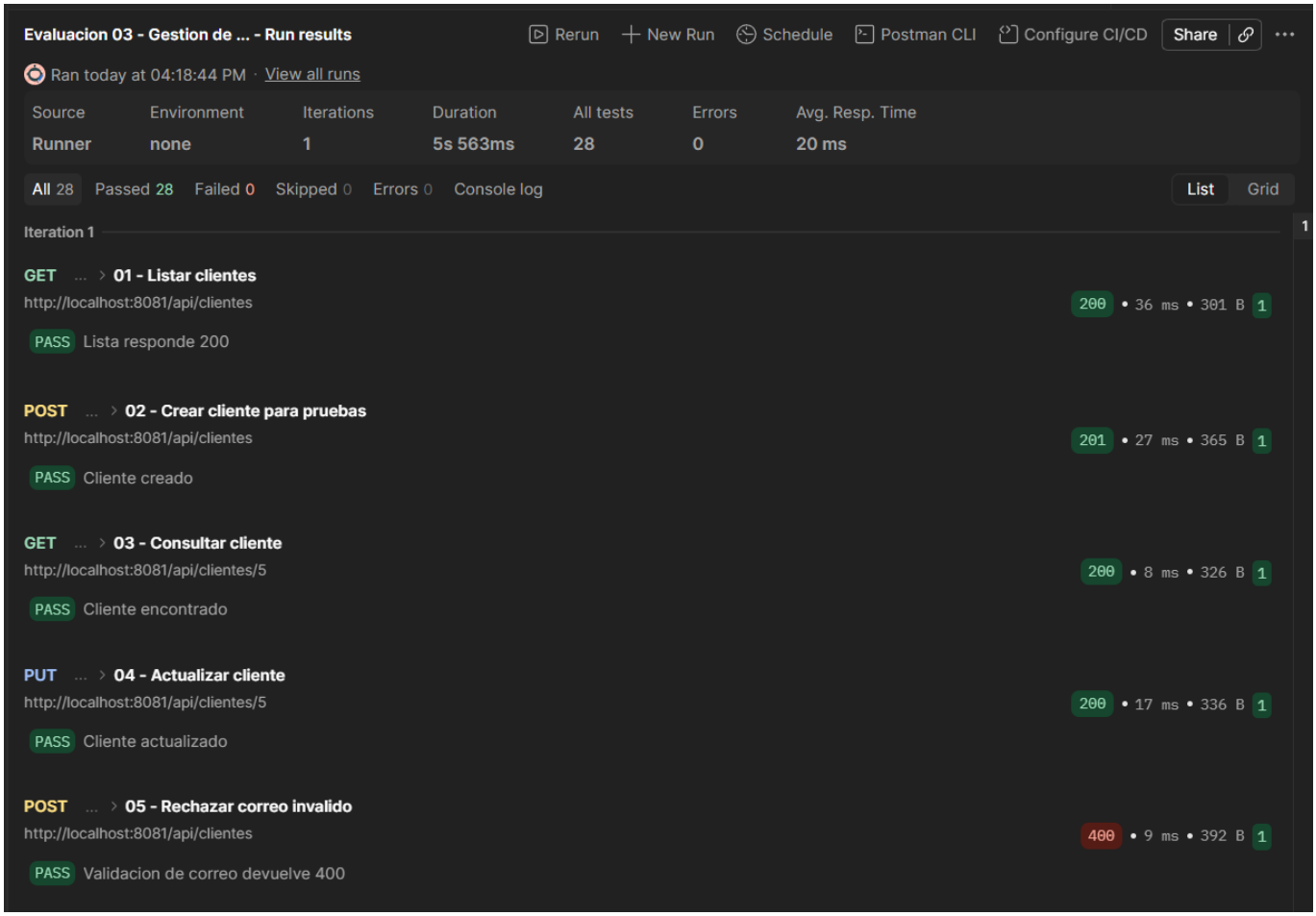
VS Code: estructura del proyecto gestion-pedidos-restaurante y los tres servicios.



XAMPP: MySQL activo en el puerto 3306. Apache permanece detenido porque no se usa para estas APIs.

Pruebas integrales - Postman

La coleccion corregida se ejecuto con el Runner de Postman Desktop. El resumen real indica 28 pruebas y 28 aprobadas, sin fallos ni errores. Los casos cubren operaciones CRUD, validaciones de entrada, referencias remotas entre servicios y limpieza de datos temporales.

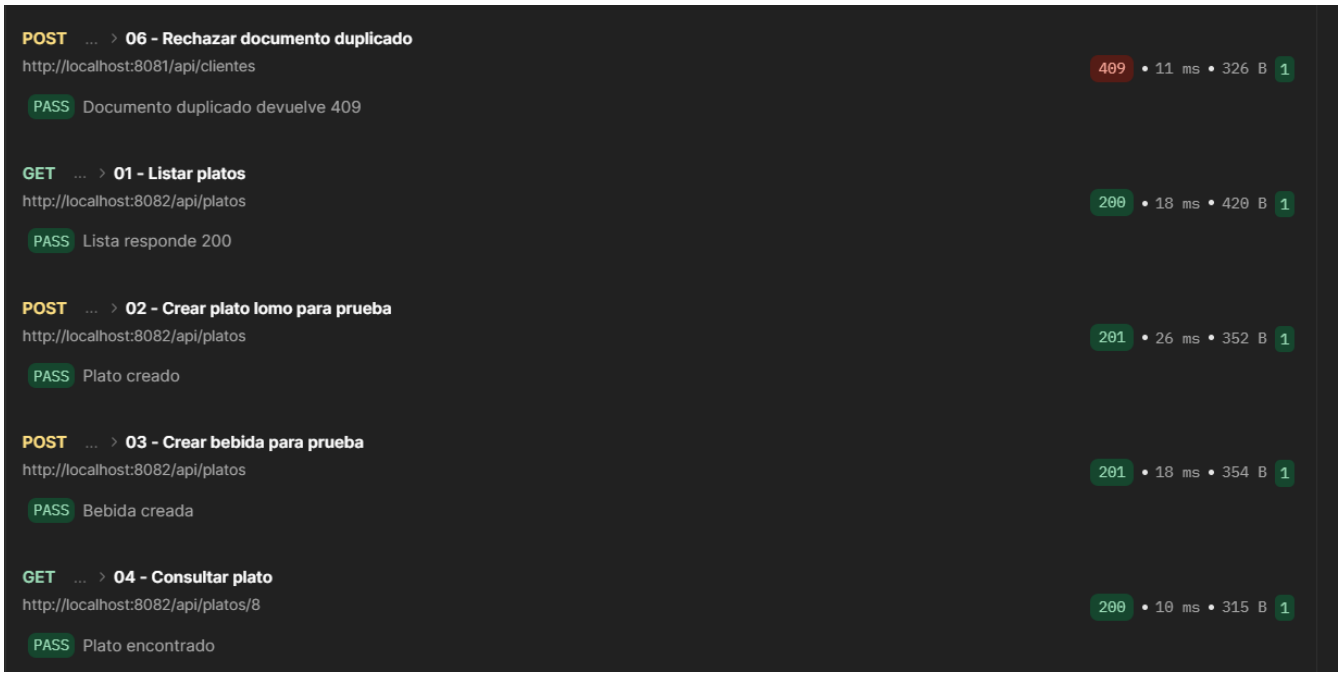


Resultado del Runner de Postman: All tests 28, Passed 28, Failed 0, Errors 0.

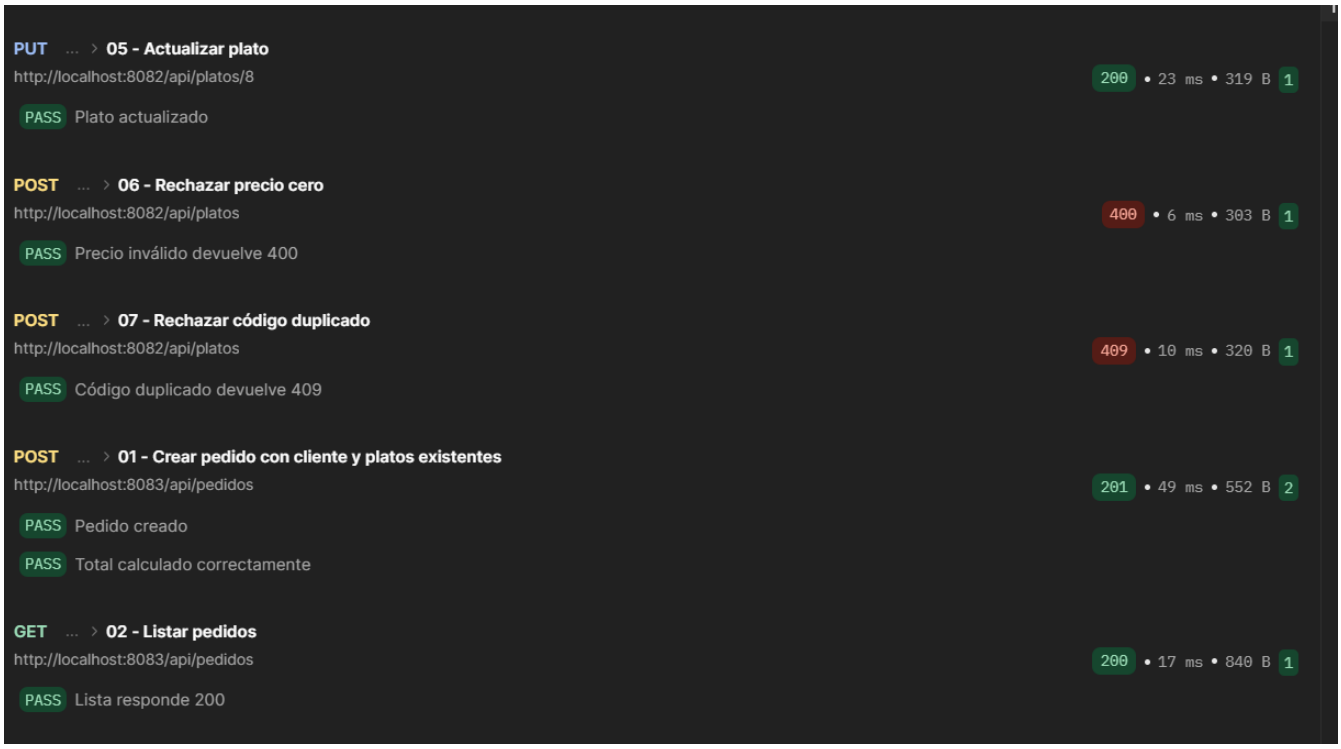
La carpeta postman/Evaluacion03_GestionPedidos.postman_collection.json contiene las solicitudes ejecutadas. Los casos negativos tambien se consideran aprobados cuando el servicio devuelve el codigo esperado (por ejemplo 400, 409, 422 o 404).

Evidencias Postman - secuencia 1

Las imagenes siguientes son capturas directas del resultado de ejecucion de Postman. La etiqueta PASS confirma que cada respuesta coincide con el codigo HTTP esperado por el caso.



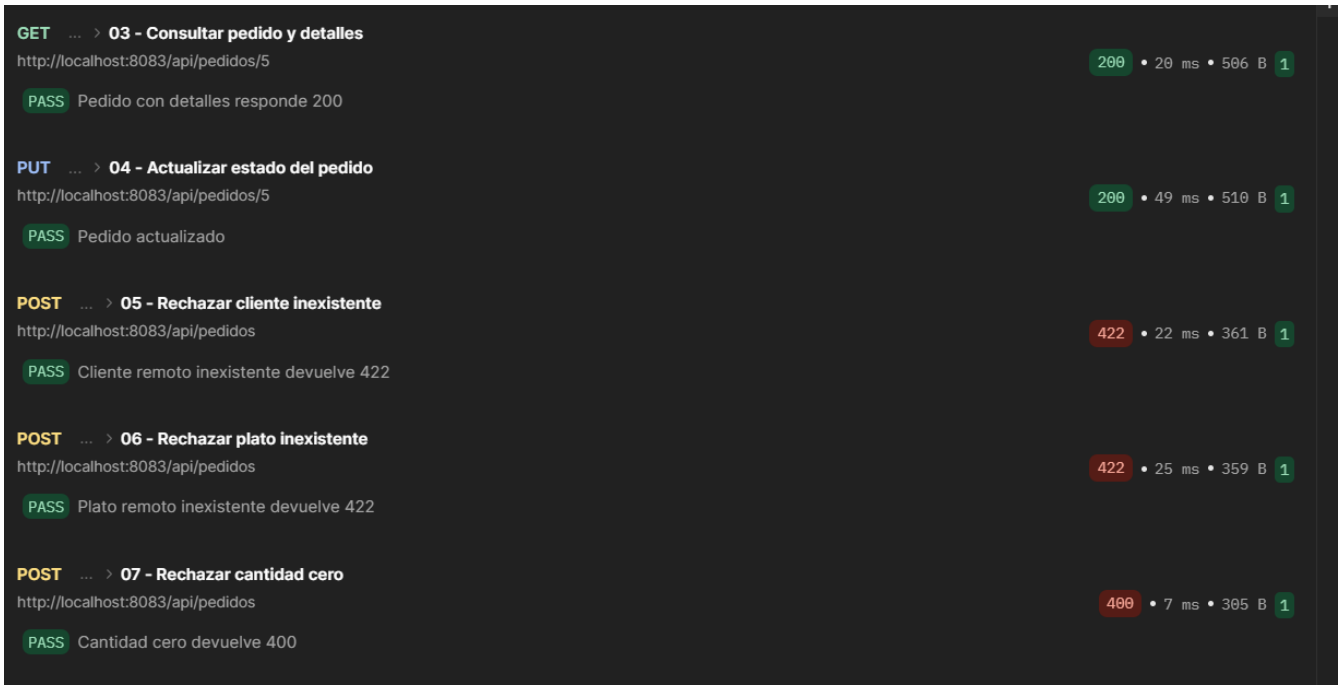
CRUD de clientes y primeras validaciones de correo/documento.



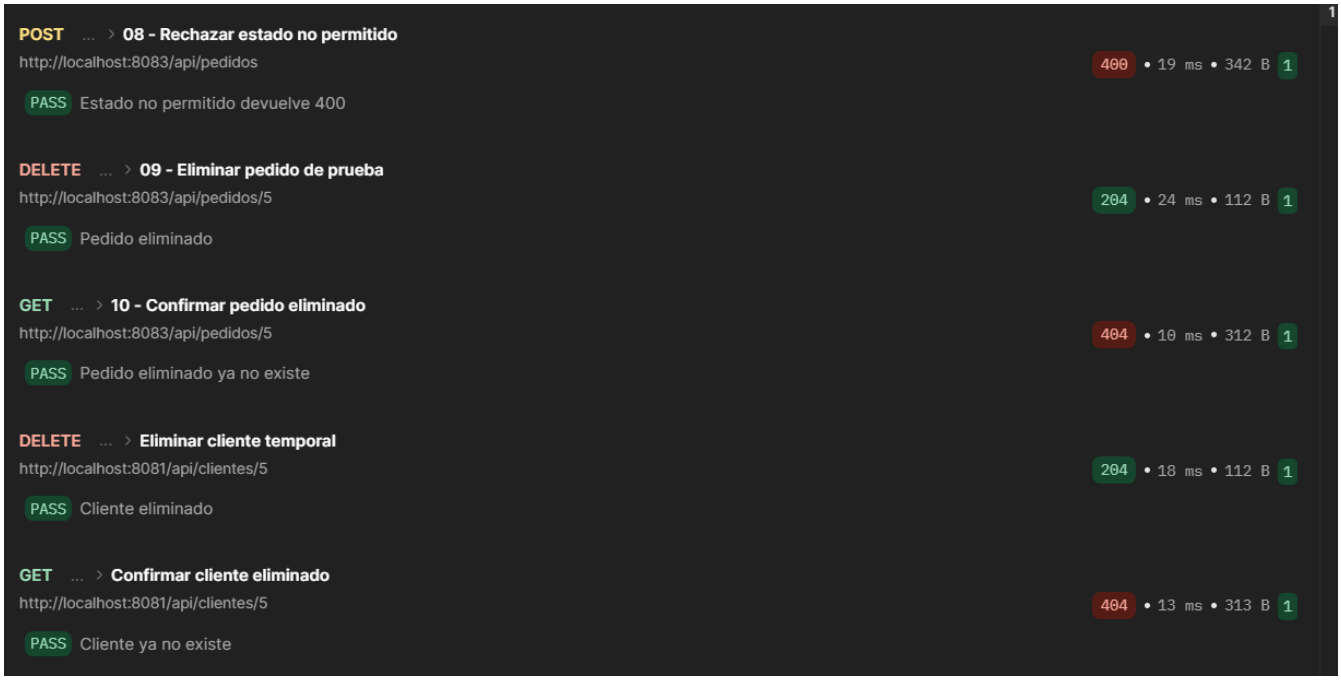
CRUD de platos, controles de precio/codigo y creacion de pedido.

Evidencias Postman - secuencia 2

Las imagenes siguientes son capturas directas del resultado de ejecucion de Postman. La etiqueta PASS confirma que cada respuesta coincide con el codigo HTTP esperado por el caso.



Consulta/actualizacion del pedido, referencias inexistentes y cantidad invalida.



Estado no permitido, eliminacion del pedido y limpieza del cliente de prueba.

Evidencias Postman - secuencia 3

Las imagenes siguientes son capturas directas del resultado de ejecucion de Postman. La etiqueta PASS confirma que cada respuesta coincide con el codigo HTTP esperado por el caso.



Limpieza de los dos platos temporales al finalizar la coleccion.

Cinco aprendizajes tecnicos - conclusiones individuales

1. Aprendi a separar responsabilidades: Cliente administra personas, Plato administra el catalogo y Pedido coordina la compra.

2. Comprendi que JPA modela relaciones dentro de una base; entre servicios independientes conviene intercambiar IDs y validar referencias por REST.

3. Aplique DTO y validaciones para controlar los datos de entrada y responder con codigos HTTP claros ante datos invalidos o duplicados.

4. Implemente el calculo de subtotales y total usando el precio vigente que entrega el servicio Plato.

5. Aprendi a separar perfiles dev y prod: en desarrollo Hibernate actualiza tablas y en produccion valida la estructura existente.

Dificultad encontrada y solucion

La dificultad tecnica fue mantener referencias consistentes cuando Cliente, Plato y Pedido guardan datos en bases separadas, sin claves foraneas entre ellas. La solucion fue guardar los IDs remotos y consultar los servicios Cliente y Plato antes de persistir el pedido; si la referencia no existe o el servicio no responde, Pedido devuelve un error HTTP explicito.

Sustentacion individual - guia de 5 a 8 minutos

Para la exposicion puedo dedicar cerca de 1 minuto al objetivo y estructura; 1 minuto a las responsabilidades de los servicios; 1 a 2 minutos a la persistencia y comunicacion REST; 2 minutos a mostrar casos validos e invalidos en Postman; y 1 minuto a mis aprendizajes y la dificultad resuelta. Debo demostrar el sistema en vivo y estar preparado para un cambio pequeno solicitado por el docente.

Correspondencia con la asignacion individual

El PDF Evaluacion 03 - D asigna a CASTILLO SUMIRE, JESUS JOSE el laboratorio individual N. 4: Pedidos de restaurante, con los microservicios Cliente, Plato y Pedido. Este informe corresponde a esa asignacion. Las referencias entre los servicios se representan mediante IDs; la configuracion de URLs se documenta en el README.

Ubicacion del proyecto

Semana12/gestion-pedidos-restaurante/ dentro del repositorio Desarrollo-de-Aplicaciones-Web. Las evidencias fuente estan en evidencias/capturas y la coleccion ejecutada esta en postman/.